ALGORITHMS · PRINTABLE WORKBOOK

Who Gets In, and Why

Stable matching (the Gale-Shapley algorithm), learned on paper: the procedure, runs you work by hand, the proofs, and 37 questions in four tiers, with an answer key that explains every option.

How to use this packet

1. Work in order. **Part 1** teaches the procedure with a worked example, then you run it yourself on warm-ups and two challenges. Every run has a blank **trace table**: one line per ask.
2. Part 1 uses plain words (ask, hold, let go, holds). From Part 2 on, the textbook words come in, each with the plain phrase it replaces. The glossary is on the Part 2 page.
3. Each question names the **skill** it checks, and every wrong option is a mistake people really make.
4. The **answer key** is at the back on its own pages. It includes a full trace for every run and explains every option. Your order of asks may differ from the key's; your final seats should not.
5. The preference lists are made up. The schools are real; their preferences are not.

Part 1	The run, by hand	9 questions
Part 2	Reading a statement exactly	7 questions
Part 3	Where the textbook chapter ends	9 questions
Part 4	The chapter’s own exercises: variants	12 questions
Summary	One page to keep	
Key	Traces, activities, every question	

PART 1 · LEARN

Ask, hold, bump, repeat

Some students, the same number of schools, one seat at each. Every student ranks every school, and every school ranks every student. We want an arrangement that **holds**: one where no student and school would *both* rather have each other than who they ended up with.

The process (students asking)

1. Pick any student who has no seat. They ask the next school on their list that they have not asked yet.
2. If that school is holding nobody, it **holds** the student. Holding is a "maybe", not a yes.
3. If it is already holding somebody, it keeps whichever of the two it likes better. The other one is **turned away** (if they were the one asking) or **let go** (if they were being held).
4. Anyone without a seat goes back to step 1, carrying on down their own list from where they left off.
5. When everybody is held, stop. Only now is anything final.

- **Turned away** costs you one name off your list. **Let go** (bumped) sends you back to spend more of your list.
- Nobody ever asks the same school twice. A school, once asked, is always holding somebody, and only ever swaps for someone it likes better.
- Either side can be the one asking. Swap the roles and run the same rules.

How to keep track on paper

Cross names off the asker's list as they ask. Under each school, write who it is holding right now, and strike them out when they are let go. Log every ask on a line of the trace table.

Worked example: three students, three seats (students asking)

Student	1st	2nd	3rd	why
Priya	MIT	UMass Amherst	NYU	*Wants the strongest engineering programme, but UMass is close to home and in-state.*
Sam	MIT	NYU	UMass Amherst	*Set on MIT, and failing that wants to be in a city.*
Ravi	NYU	MIT	UMass Amherst	*Set on New York. Everything else is a fallback.*

School	1st	2nd	3rd	why
MIT	Sam	Ravi	Priya	*Weighting research experience above everything else.*
UMass Amherst	Ravi	Priya	Sam	*Building out its data science cohort.*
NYU	Priya	Sam	Ravi	*Looking for leadership and portfolio work.*

#	asks	whom	what happens
1	Priya	MIT	MIT was holding nobody, so holds Priya.
2	Sam	MIT	MIT likes Sam better than Priya: holds Sam, **lets Priya go**.
3	Priya	UMass Amherst	UMass Amherst was holding nobody, so holds Priya.
4	Ravi	NYU	NYU was holding nobody, so holds Ravi.

Result: Priya → **UMass Amherst** · Sam → **MIT** · Ravi → **NYU**. Priya was held by MIT and then bumped by Sam, whom MIT likes best. Nobody got everything: Priya is at her 2nd choice. Does it hold? Priya would rather have MIT, but MIT has Sam, its first choice, so MIT would not switch. It takes two. Sam and Ravi are at their first choices. So nobody can break it.

PART 1 · TRY IT

Run it the other way

WARM-UP W1 · YOUR TURN

Same three students and schools as the worked example, but now the **schools** ask (each school walks down its own list of students; each student holds the best school that has asked so far).

#	Who asks	Whom	What happens (held? turned away? who is let go?)
1			
2			
3			
4			
5			

Final seats: Priya → _____ Sam → _____ Ravi → _____

Compare with the worked example. Who does better, and who does worse?

Checking whether an arrangement holds

Two people **break** an arrangement if they would both rather have each other than who they have. To check an arrangement, go student by student:

1. List the schools this student ranks *above* the one they have.
2. For each of those, ask: does that school rank this student above the student it has?
3. If yes, those two would both switch. The arrangement is broken. If no student has such a school, it holds.

Student	has	schools they'd rather have	would any of those rather have them?
Priya			
Sam			
Ravi			

Use this table on the result of W1. Does it hold?

Three facts worth carrying into the questions (proved in Part 3):

1. The order in which the asks happen never changes the ending, only the story.
2. Whichever side asks gets the best seat for them that any arrangement that holds can give; the other side gets its worst.
3. So anyone in the same seat in both runs is in that seat in *every* arrangement that holds.

PART 1 · MORE PRACTICE

Warm-ups before the questions

WARM-UP W2 · TWO STUDENTS WHO WANT OPPOSITE THINGS

Two students who want opposite things. Run it both ways.

Student	1st	2nd	why
Priya	MIT	NYU	*MIT first, and she is not really considering anywhere else.*
Sam	NYU	MIT	*New York first. MIT would be a consolation.*

School	1st	2nd	why
MIT	Sam	Priya	*Sam is their standout applicant this round.*
NYU	Priya	Sam	*Priya is exactly who they are looking for.*

#	Who asks	Whom	What happens (held? turned away? who is let go?)
1			
2			
3			

Final seats: Priya → _____ Sam → _____

Schools asking:

Final seats: Priya → _____ Sam → _____

WARM-UP W3 · EVERYONE WANTS THE SAME THINGS

Everyone wants the same things. Run it with the students asking and count the asks. Then predict the schools-asking result before running it.

Student	1st	2nd	3rd	4th
Priya	MIT	Berkeley	UMass Amherst	Northeastern
Sam	MIT	Berkeley	UMass Amherst	Northeastern
Ravi	MIT	Berkeley	UMass Amherst	Northeastern
Maya	MIT	Berkeley	UMass Amherst	Northeastern

School	1st	2nd	3rd	4th
MIT	Priya	Sam	Ravi	Maya
Berkeley	Priya	Sam	Ravi	Maya
UMass Amherst	Priya	Sam	Ravi	Maya
Northeastern	Priya	Sam	Ravi	Maya

Trace, students asking (run the schools-asking side on scrap paper, or predict it):

#	Who asks	Whom	What happens (held? turned away? who is let go?)
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			

Students asking:

Final seats: Priya → _____ Sam → _____ Ravi → _____

Maya → _____

Schools asking:

Final seats: Priya → _____ Sam → _____ Ravi → _____

Maya → _____

WARM-UP W4 · THE TIME IT MAKES NO DIFFERENCE

Preferences genuinely conflict here. Run it both ways and compare.

Student	1st	2nd	3rd	4th
Priya	MIT	Berkeley	NYU	UMass Amherst
Sam	NYU	Berkeley	MIT	UMass Amherst
Ravi	NYU	Berkeley	MIT	UMass Amherst
Maya	MIT	Berkeley	UMass Amherst	NYU

School	1st	2nd	3rd	4th
MIT	Maya	Ravi	Sam	Priya
UMass Amherst	Priya	Maya	Sam	Ravi
NYU	Priya	Maya	Sam	Ravi
Berkeley	Maya	Sam	Ravi	Priya

Trace, students asking (run the schools-asking side on scrap paper, or predict it):

#	Who asks	Whom	What happens (held? turned away? who is let go?)
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			

Students asking:

Final seats: Priya → _____ Sam → _____ Ravi → _____

Maya → _____

Schools asking:

Final seats: Priya → _____ Sam → _____ Ravi → _____

Maya → _____

PART 1 · CHALLENGE

Two harder instances

CHALLENGE C1 · ONE QUESTION, SEVEN PEOPLE MOVED

Five and five. Run it with the students asking. Count how many people get let go, and notice how late in the run the chain happens. Then run it with the schools asking.

Student	1st	2nd	3rd	4th	5th
Priya	UMass Amherst	Michigan	MIT	NYU	Berkeley
Sam	Michigan	UMass Amherst	NYU	MIT	Berkeley
Ravi	MIT	UMass Amherst	Berkeley	NYU	Michigan
Maya	Berkeley	MIT	UMass Amherst	Michigan	NYU
Diego	MIT	Berkeley	UMass Amherst	Michigan	NYU

School	1st	2nd	3rd	4th	5th
MIT	Maya	Priya	Diego	Sam	Ravi
UMass Amherst	Diego	Maya	Sam	Ravi	Priya
NYU	Priya	Diego	Ravi	Sam	Maya
Berkeley	Sam	Priya	Diego	Ravi	Maya
Michigan	Ravi	Priya	Maya	Sam	Diego

#	Who asks	Whom	What happens (held? turned away? who is let go?)
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			

Final seats: Priya → _____ Sam → _____ Ravi → _____

Maya → _____ Diego → _____

Schools asking:

Final seats: Priya → _____ Sam → _____ Ravi → _____

Maya → _____ Diego → _____

Bonus: this instance has four arrangements that hold. Find the two that neither run produces.

CHALLENGE C2 · TWO WHO EVERYBODY WANTS, TWO WHO NOBODY DOES

MIT and UMass Amherst are the "good" schools and Priya and Sam the "good" students: every list ranks both good people above both bad ones. Find **every** arrangement that holds (there are several). What do they all have in common?

Student	1st	2nd	3rd	4th
Priya	MIT	UMass Amherst	NYU	Berkeley
Sam	UMass Amherst	MIT	Berkeley	NYU
Ravi	MIT	UMass Amherst	Berkeley	NYU
Maya	UMass Amherst	MIT	NYU	Berkeley

School	1st	2nd	3rd	4th
MIT	Sam	Priya	Ravi	Maya
UMass Amherst	Priya	Sam	Maya	Ravi
NYU	Priya	Sam	Ravi	Maya
Berkeley	Sam	Priya	Maya	Ravi

PART 1 · QUESTIONS

Tier 1: The run, by hand

1.1 *Skill: working a run by hand*

Four students, four seats, and the students are asking. Where does everybody end up?

Student	1st	2nd	3rd	4th	why
Priya	Berkeley	UMass Amherst	NYU	MIT	*Set on Berkeley for one particular professor. MIT is the one place she would rather not go.*
Sam	MIT	Berkeley	UMass Amherst	NYU	*MIT or nothing, really. After that he is ranking by how much fieldwork each one offers.*
Ravi	MIT	UMass Amherst	NYU	Berkeley	*Ranking strictly by distance from home in Massachusetts, nearest first.*
Maya	Berkeley	NYU	MIT	UMass Amherst	*Wants a city and a coast, in that order.*

School	1st	2nd	3rd	4th	why
MIT	Priya	Sam	Ravi	Maya	*Weighting research experience above everything else.*
UMass Amherst	Sam	Priya	Maya	Ravi	*Looking for students who have already published something.*
NYU	Priya	Ravi	Sam	Maya	*Leadership and portfolio work.*
Berkeley	Sam	Priya	Ravi	Maya	*Weighting maths preparation above everything else.*

#	Who asks	Whom	What happens (held? turned away? who is let go?)
1			
2			
3			
4			
5			
6			
7			
8			

Final seats: Priya → _____ Sam → _____ Ravi → _____

Maya → _____

1.2 *Skill: whether the answer depends on who asks*

Same four people, same lists, but now the schools are asking. Where does everybody end up?

Same lists as question 1.1.

Student	1st	2nd	3rd	4th
Priya	Berkeley	UMass Amherst	NYU	MIT
Sam	MIT	Berkeley	UMass Amherst	NYU
Ravi	MIT	UMass Amherst	NYU	Berkeley
Maya	Berkeley	NYU	MIT	UMass Amherst

School	1st	2nd	3rd	4th
MIT	Priya	Sam	Ravi	Maya
UMass Amherst	Sam	Priya	Maya	Ravi
NYU	Priya	Ravi	Sam	Maya
Berkeley	Sam	Priya	Ravi	Maya

#	Who asks	Whom	What happens (held? turned away? who is let go?)
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			

Final seats: Priya → _____ Sam → _____ Ravi → _____

Maya → _____

1.3 *Skill: what makes an arrangement hold*

Here are six ways to seat these four students. Tick every one that cannot be broken by two people who would both rather have each other.

Same lists as question 1.1.

Student	1st	2nd	3rd	4th
Priya	Berkeley	UMass Amherst	NYU	MIT
Sam	MIT	Berkeley	UMass Amherst	NYU
Ravi	MIT	UMass Amherst	NYU	Berkeley
Maya	Berkeley	NYU	MIT	UMass Amherst

School	1st	2nd	3rd	4th
MIT	Priya	Sam	Ravi	Maya
UMass Amherst	Sam	Priya	Maya	Ravi
NYU	Priya	Ravi	Sam	Maya
Berkeley	Sam	Priya	Ravi	Maya

A ☐ holds `Priya → Berkeley · Sam → MIT · Ravi → UMass Amherst · Maya → NYU`
If not, who would both switch? _____ & _____

B ☐ holds `Priya → Berkeley · Sam → UMass Amherst · Ravi → MIT · Maya → NYU`
If not, who would both switch? _____ & _____

C ☐ holds `Priya → Berkeley · Sam → MIT · Ravi → NYU · Maya → UMass Amherst`
If not, who would both switch? _____ & _____

D ☐ holds `Priya → MIT · Sam → Berkeley · Ravi → UMass Amherst · Maya → NYU`
If not, who would both switch? _____ & _____

E ☐ holds `Priya → NYU · Sam → MIT · Ravi → UMass Amherst · Maya → Berkeley`
If not, who would both switch? _____ & _____

F ☐ holds `Priya → UMass Amherst · Sam → NYU · Ravi → Berkeley · Maya → MIT`
If not, who would both switch? _____ & _____

1.4 *Skill: what being in the same seat twice means*

About the four-by-four lists in question 1.1.

Priya ends at Berkeley and Sam at MIT whichever side does the asking. What does that tell you about them?

◯ **A.** They are in the same seats in every arrangement that holds here, not only in those two runs.

◯ **B.** They were lucky with the order the questions happened to be asked in.

◯ **C.** Berkeley and MIT are the two strongest schools, so they settle first.

◯ **D.** This set of lists has exactly one arrangement that holds.

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.5 *Skill: whether the second answer is a second answer*

About the four-by-four lists in question 1.1.

A worksheet answer runs the process with the students asking, and then answers "find another arrangement that holds" with "have the students be the ones who ask". What is wrong with it?

◯ **A.** Nothing, as long as it also writes out what the arrangement is.

◯ **B.** Flipping which side asks never produces a different arrangement.

◯ **C.** That is the run it has already done. It hands back the arrangement it started from.

◯ **D.** A second arrangement has to be found by hand, because the process only ever produces one.

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.6 *Skill: whether the route changes the ending*

About the four-by-four lists in question 1.1.

Two people work the same set of lists with the same side asking, and pick a different person to ask next at every stage. What happens?

◯ **A.** They get different endings, which is why a worksheet has to fix an order.

◯ **B.** They get the same ending only if nobody is displaced on the way.

◯ **C.** They get the same ending only when exactly one arrangement holds.

◯ **D.** They get the same ending. Which question comes next changes the story and not the result.

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.7 *Skill: working the lecture's own example by hand*

The lecture's running example (slide 25), with its colleges a, b, c named MIT, UMass Amherst and NYU and its students 1, 2, 3 named Priya, Sam and Ravi. The lecture has the colleges ask, so here the schools are asking. Where does everybody end up?

Student	1st	2nd	3rd
Priya	NYU	MIT	UMass Amherst
Sam	MIT	UMass Amherst	NYU
Ravi	MIT	UMass Amherst	NYU

School	1st	2nd	3rd
MIT	Priya	Sam	Ravi
UMass Amherst	Sam	Priya	Ravi
NYU	Priya	Ravi	Sam

#	Who asks	Whom	What happens (held? turned away? who is let go?)
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			

Final seats: Priya → _____ Sam → _____ Ravi → _____

1.8 *Skill: finding the two people who would both switch*

The lists from the lecture's first clicker (slide 22), named the same way. Tick every seating that cannot be broken by two people who would both rather have each other.

Student	1st	2nd	3rd
Priya	UMass Amherst	MIT	NYU
Sam	MIT	UMass Amherst	NYU
Ravi	MIT	UMass Amherst	NYU

School	1st	2nd	3rd
MIT	Priya	Sam	Ravi
UMass Amherst	Sam	Priya	Ravi
NYU	Priya	Sam	Ravi

A ☐ holds `Priya → MIT · Sam → NYU · Ravi → UMass Amherst`
If not, who would both switch? _____ & _____

B ☐ holds `Priya → MIT · Sam → UMass Amherst · Ravi → NYU`
If not, who would both switch? _____ & _____

C ☐ holds `Priya → NYU · Sam → MIT · Ravi → UMass Amherst`
If not, who would both switch? _____ & _____

D ☐ holds `Priya → UMass Amherst · Sam → MIT · Ravi → NYU`
If not, who would both switch? _____ & _____

1.9 *Skill: running the process when every first ask sticks*

Homework 1, question 1, named the same way, with the colleges (the schools) asking. Where does everybody end up?

Student	1st	2nd	3rd
Priya	UMass Amherst	MIT	NYU
Sam	MIT	UMass Amherst	NYU
Ravi	NYU	UMass Amherst	MIT

School	1st	2nd	3rd
MIT	Priya	Sam	Ravi
UMass Amherst	Sam	Priya	Ravi
NYU	Ravi	Sam	Priya

#	Who asks	Whom	What happens (held? turned away? who is let go?)
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			

Final seats: Priya → _____ Sam → _____ Ravi → _____

PART 2 · LEARN

Why this works, and the textbook words

Three things are true of every finished run. Each comes plainly first, then the way a textbook says it.

1. It finishes

Every ask uses up one name off one list, and nobody ever asks the same name twice. With n askers and n names on each list, there can be at most n × n asks, and then there is nothing left to ask.

Textbook: The algorithm terminates after at most n^2 proposals. Each proposer keeps a cursor into their preference list; each proposal advances it by one and it never moves back (it is monotone).

2. Nobody is left out

Once anybody asks a school, that school is holding somebody forever after. So if some student had asked every school, every school would be holding somebody: that is n students held, and there are only n students. The one who ran out would have to be one of them. Contradiction.

Textbook: The resulting matching is perfect. A receiver that has been proposed to is never again unmatched…

3. It holds

Take a student and a school not together, where the student would rather have that school. Then the student asked it at some point (they walked past it on their list to get where they are). From that ask on the school was holding somebody and only ever traded up. So it ends with somebody it likes better than this student and would not switch. It takes two.

Textbook: The resulting matching is stable: it admits no blocking pair. Uses the invariant that a receiver only ever trades up its own list.

Glossary: textbook word = plain phrase

Textbook	Plain
instance	one complete set of lists to work with
matching	an arrangement, who ended up with whom
stable	holds, cannot be broken
blocking pair	two people who would both rather have each other
propose	ask
proposer	whoever is doing the asking
receiver	whoever is being asked
reject	turn away
optimal	the best anybody on this side can do here
pessimal	the worst anybody on this side can do here
algorithm	the process
perfect	nobody is left out, everybody on both sides ends up with somebody
maximum matching	an arrangement with as many pairs in it as any arrangement can have, whatever anybody wants
valid partner	somebody you are paired with in at least one arrangement that holds
dominant strategy	the best list to hand in, whatever everybody else hands in
$O(n^2)$	work that grows like n times n, give or take a fixed multiple
strong instability	two people not together who each strictly prefer the other to who they have
weak instability	two people not together where one strictly prefers the other, and the other prefers them too or does not mind
NP-hard	one of the problems nobody knows how to solve quickly
college (c), student (s)	the textbook's names for the two sides; "college" = school

PART 2 · LEARN

Reading a statement exactly

Problem sets grade whether you read a claim precisely. The tier's statement (Kleinberg & Tardos ch. 1, exercise 1):

> ```
> In every instance of the Stable Matching Problem, there is a stable matching containing a pair (c, s)
> such that c is ranked first on the preference list of s and s is ranked first on the preference list of
> c.
> ```

Symbols

∀ for every · ∃ there is at least one · ∧ and · ¬ it is not the case that. An **instance** is one complete set of inputs: the two sides and a strict ranking of the whole other side for each person. It does not include who asks, and it does not include any answer.

English phrases are quantifiers

Underline them in order and the formula writes itself: "in every instance" = ∀I, "there is a stable matching" = ∃M, "containing a pair" = ∃(c, s) ∈ M.

Negating: walk left to right

Every quantifier turns into the other kind, nothing moves, and the part at the end is negated. ∀I ∃M ∃(c,s): φ becomes ∃I ∀M ∀(c,s): ¬φ.

> **What a disproof must do.** The negation says: *there is an instance* in which *every* stable matching lacks the pair. Finding the instance is half. Ruling out *every* stable matching of it is the other half, and it is the half that gets skipped.

TRY IT

1. Underline the three quantifier phrases in the statement above. 2. Write its negation in plain English. 3. In the instance below, list every matching, say which are stable, and check each for a pair who are first on each other's lists.

```
c1 : s1 > s2        s1 : c2 > c1
c2 : s2 > s1        s2 : c1 > c2
```

PART 2 · QUESTIONS

Tier 2: Reading a statement exactly

Glossary on the Part 2 Learn page. c = college (school), s = student.

2.1 *Skill: whether a stable matching is unique*

Lecture 1, slide 24: colleges a: 1 2 and b: 2 1; students 1: b a and 2: a b. The lecture runs propose-and-reject with the colleges proposing, which on this page is the schools doing the asking. Which matching is stable? (Here a, b are MIT and NYU and 1, 2 are Priya and Sam: these are the lists shown above.)

- ◯ **A.** {(a, 1), (b, 2)} only
- ◯ **B.** {(a, 2), (b, 1)} only
- ◯ **C.** Neither
- ◯ **D.** Both

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.2 *Skill: what the statement is quantifying over*

> `In every instance of the Stable Matching Problem, there is a stable matching containing a pair (c, s) such that c is ranked first on the preference list of s and s is ranked first on the preference list of c.`

Before anything else: what is an instance of the Stable Matching Problem?

- ◯ **A.** A set of stable matchings, together with which side proposes and which side accepts.
- ◯ **B.** One complete set of inputs: the colleges, the students, and a strict ranking of the whole other side for each of them.
- ◯ **C.** The algorithm, applied to a particular set of preference lists.
- ◯ **D.** A single matching of colleges to students.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.3 *Skill: turning English into quantifiers*

> `In every instance of the Stable Matching Problem, there is a stable matching containing a pair (c, s) such that c is ranked first on the preference list of s and s is ranked first on the preference list of c.`

Let I be an instance and M a stable matching. Read first(c, s) as "c is ranked first on the list of s", so it is c that sits at the top, and s whose list it sits at the top of. Three symbols do the work: ∀ means "for every", ∃ means "there is at least one", and ∧ means "and". Which formula says this?

- ◯ **A.** ∀I ∃M ∀(c, s) ∈ M : first(c, s) ∧ first(s, c)
- ◯ **B.** ∀I ∃M ∃(c, s) ∈ M : first(c, s) ∧ first(s, c)
- ◯ **C.** ∀I ∀M ∃(c, s) ∈ M : first(c, s) ∧ first(s, c)
- ◯ **D.** ∃I ∃M ∀(c, s) ∈ M : first(c, s) ∧ first(s, c)

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.4 *Skill: negating a statement with three quantifiers*

```
The statement is  ∀I  ∃M  ∃(c, s) ∈ M :  φ,  where φ is  first(c, s) ∧ first(s, c).
```

Two more symbols: ¬ means "it is not the case that", and φ is only a name for the long part at the end, so the shape of the sentence is easier to see. Tick every correct negation. There is more than one.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** ∃I ∀M ¬(∃(c, s) ∈ M : φ)
- ☐ **B.** ∀I ∃M ∀(c, s) ∈ M : ¬φ
- ☐ **C.** ∃I ∀M ∀(c, s) ∈ M : ¬φ
- ☐ **D.** ∃I ∃M ∀(c, s) ∈ M : ¬φ
- ☐ **E.** ∃I ∀M ∃(c, s) ∈ M : ¬φ
- ☐ **F.** ∀I ∀M ∀(c, s) ∈ M : ¬φ

2.5 *Skill: the same negation, in words*

```
In every instance of the Stable Matching Problem, there is a stable matching containing a pair (c, s)
such that c is ranked first on the preference list of s and s is ranked first on the preference list of
c.
```

Which sentence is the negation of the statement above?

- ◯ **A.** In every instance, there is no stable matching containing such a pair.
- ◯ **B.** In every instance, there is a stable matching containing no pair (c, s) with c first on the list of s and s first on the list of c.
- ◯ **C.** There is an instance in which every stable matching contains no pair (c, s) with c first on the list of s and s first on the list of c.
- ◯ **D.** There is an instance with a stable matching containing no pair (c, s) with c first on the list of s and s first on the list of c.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.6 *Skill: how much a counterexample has to rule out*

```
"The statement is false. Consider the instance
    c1 : s1 > s2        s1 : c2 > c1
    c2 : s2 > s1        s2 : c1 > c2
In the stable matching M = {(c1, s1), (c2, s2)}, neither student has their
first choice, so M does not include a pair (c, s) where c is ranked first
on the list of s and s is ranked first on the list of c."
```

Remember first(c, s) means c sits at the top of the list belonging to s. The instance here is a good one and the conclusion drawn from it is right. What has the argument not shown?

- ◯ **A.** It checked one stable matching. The statement says some stable matching contains such a pair, so refuting it means ruling out every stable matching of the instance, and this one has two.
- ◯ **B.** Its reasoning about M is wrong, because M is not stable.
- ◯ **C.** Whether the students have their first choice is beside the point, because the claim is about the colleges.
- ◯ **D.** Nothing. It exhibits an instance and a matching with no such pair, which is what the negation asks for.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.7 *Skill: building the disproof that the flawed answer nearly had*

> `In every instance of the Stable Matching Problem, there is a stable matching containing a pair (c, s) such that c is ranked first on the preference list of s and s is ranked first on the preference list of c.`

Build the proof that the statement is false.

LINE 1 · WHAT HAS TO BE PRODUCED

- ◯ **A.** An instance in which every stable matching lacks a pair that is first on both lists.
- ◯ **B.** An instance with a stable matching that lacks such a pair.
- ◯ **C.** A proof that no instance has a stable matching containing such a pair.

LINE 2 · THE INSTANCE

- ◯ **A.** Two colleges and two students who agree on everything: c1 : s1 > s2, c2 : s1 > s2, s1 : c1 > c2, s2 : c1 > c2.
- ◯ **B.** One college and one student.
- ◯ **C.** Two colleges and two students, with c1 : s1 > s2, c2 : s2 > s1, s1 : c2 > c1, s2 : c1 > c2.

LINE 3 · WHAT TO CHECK

- ◯ **A.** Every stable matching of the instance. There are two: {(c1, s1), (c2, s2)} and {(c1, s2), (c2, s1)}.
- ◯ **B.** The matching the propose and reject algorithm returns, which is stable by the theorem.
- ◯ **C.** Every matching, stable or not.

LINE 4 · THE CHECK

- ◯ **A.** Neither gives a student their first choice, so neither can contain such a pair.
- ◯ **B.** Neither contains a mutual first choice, because no pair in this instance is first on both lists at all: c1 tops the list of s2 while c1 wants s1, and c2 tops the list of s1 while c2 wants s2.
- ◯ **C.** The first contains no such pair, and the second is not stable.

LINE 5 · THE CONCLUSION

- ◯ **A.** So this instance has no stable matching containing a pair first on both lists. That is the negation, so the statement is false.
- ◯ **B.** So no instance has a stable matching containing such a pair, and the statement is false.
- ◯ **C.** So the propose and reject algorithm does not always find such a pair, and the statement is false.

One line per step. The proof counts only if every line is right.

PART 3 · LEARN

Running time, and who comes out best

How many asks?

At most n^2: each ask spends a name from a list and names are never put back. (The exact worst case is $n^2 - n + 1$.) A chain of bumps can be long but never exponential, because each bump still spends a name.

Making each ask cost O(1)

When a school holding somebody is asked, it compares two names on its list. Scanning the list each time costs O(n), making the whole run $O(n^3)$. Fix: before the run, build for each school an array **ranking[student] = position on this school's list**. That costs $O(n^2)$ once, and afterwards every comparison is two array reads. Keep free askers in a queue or stack so picking the next is O(1) too.

ACTIVITY · BUILD THE RANKING ARRAY FOR THE FOUR-BY-FOUR INSTANCE

School lists: **MIT**: Priya > Sam > Ravi > Maya · **UMass Amherst**: Sam > Priya > Maya > Ravi · **NYU**: Priya > Ravi > Sam > Maya · **Berkeley**: Sam > Priya > Ravi > Maya

ranking	Priya	Sam	Ravi	Maya
MIT				
UMass Amherst				
NYU				
Berkeley				

Using only your table: UMass Amherst is holding Ravi and Maya asks. What happens?

Best for the askers, worst for the asked

Call a school a **valid partner** of a student if at least one stable matching pairs them. The theorem: **no asker is ever turned away by a valid partner**. Proof idea: look at the *first* time it happens, p rejected by valid partner r in favour of q. Because it is the first time, q has not been rejected by any valid partner, so q likes r at least as much as any valid partner, including q's partner in the stable matching M that pairs p with r. Then q and r block M. Contradiction. So every asker ends with their **best valid partner** (proposer-optimal) and, mirror image, every receiver with its **worst** (receiver-pessimal). This is also why the order of asks cannot matter.

Does lying pay?

Not for the askers: telling the truth is a **dominant strategy** for them (Dubins & Freedman; Roth). The receivers *can* sometimes gain by misreporting, e.g. truncating their list. No process that always ends stable can make honesty best for both sides.

Which assumptions carry weight?

Lists are **strict** (no ties) and **complete** (everyone ranks everyone). Allow incomplete lists and people can end unmatched, so "nobody is left out" fails and stability must be restated to include "nobody" as an option. It still finishes, and the result is still stable under the new definition.

PART 3 · QUESTIONS

Tier 3: Where the textbook chapter ends

3.1 *Skill: the counting argument behind the running time*

With n on each side, what is the ceiling on how many questions the process can ask? Not the exact record, the number you can be sure it never passes.

- ◯ **A.** n, because everybody asks once.
- ◯ **B.** n^2, because each proposer walks down a list of n names and never asks the same name twice.
- ◯ **C.** n!, since that is how many matchings there are.
- ◯ **D.** 2^n in the worst case, because one person being let go can set off a chain.

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.2 *Skill: counting the asks in one run*

The lecture's running example, with the schools asking. How many asks does the run make, first to last?

- ◯ **A.** 3
- ◯ **B.** 4
- ◯ **C.** 6
- ◯ **D.** 9

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.3 *Skill: counting the asks in one run*

About the four-by-four lists in question 1.1.

The four-by-four from tier 1, with the schools asking. How many asks does the run make?

- ◯ **A.** 4
- ◯ **B.** 8
- ◯ **C.** 16
- ◯ **D.** 9

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.4 *Skill: what the number of asks depends on*

These lists have exactly one stable matching, so both runs end in the same place. How many asks does each run make?

- ◯ **A.** The same both ways, 10, since both runs end in the same place.
- ◯ **B.** 10 with the students asking, 8 with the schools asking.
- ◯ **C.** 8 with the students asking, 10 with the schools asking.
- ◯ **D.** 16 both ways: four a side, four asks each.

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.5 *Skill: what an $O(n^2)$ bound rests on besides the procedure*

When a receiver is already holding somebody, the ask ends with it comparing two names on its own list. Written the obvious way, it reads down its list each time to find them, and the whole run then costs about n times n times n units of work. Textbooks write that $O(n^3)$, and $O(n^2)$ for n times n. What brings this run down to $O(n^2)$?

- ◯ **A.** Sorting each preference list once at the start.
- ◯ **B.** Keeping the proposers who are free in a queue rather than a list.
- ◯ **C.** For each receiver, an array giving the position of every proposer on its list, built once before the run.
- ◯ **D.** Nothing does. Comparing two names on a list of length n is inherently O(n).

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.6 *Skill: the proof that the asking side does best*

```
Call a receiver r a valid partner of proposer p if there is at least one stable
matching that puts them together. It need not be the one the process finds.
Claim: no proposer is ever rejected by a valid partner.
```

Build the proof.

LINE 1 · THE SETUP

- ◯ **A.** Suppose not, and consider the first moment in the whole run at which some proposer p is rejected by a valid partner r.
- ◯ **B.** Suppose not, and consider any moment at which some proposer p is rejected by a valid partner r.
- ◯ **C.** Suppose not, and consider the last moment at which some proposer p is rejected by a valid partner r.

LINE 2 · WHAT THE REJECTION GIVES YOU

- ◯ **A.** r rejected p because p had already been rejected by everybody above r on p's list.
- ◯ **B.** r rejected p in favour of some other proposer q that r prefers, and let M be a stable matching pairing p with r.
- ◯ **C.** r rejected p because r is p's last choice.

LINE 3 · WHAT Q PREFERS

- ◯ **A.** q prefers r to their own partner in M, because q has not yet been rejected by any valid partner, so every receiver q passed over on the way to r was not a valid partner for q.
- ◯ **B.** q prefers r to their own partner in M, because r is holding q and the process only ever improves a proposer's position.
- ◯ **C.** q prefers r to their own partner in M, because M is stable and stable matchings give everybody their first choice.

LINE 4 · THE CONTRADICTION

- ◯ **A.** Then p and r are a blocking pair in M, since p was rejected by r.
- ◯ **B.** Then M pairs q with r, which contradicts M pairing p with r.
- ◯ **C.** Then q and r are a blocking pair in M: r prefers q to p, who is r's partner in M, and q prefers r to their own partner in M. So M is not stable.

LINE 5 · WHAT FOLLOWS

- ◯ **A.** No proposer is ever rejected by a valid partner, so every proposer ends with the best valid partner they have, and the outcome does not depend on the order the asks happened in.
- ◯ **B.** Every proposer ends with their first choice.
- ◯ **C.** Every proposer does at least as well as every receiver.

One line per step. The proof counts only if every line is right.

3.7 *Skill: what holds of every instance rather than of this one*

Tick every statement that is true of every instance, not only of the ones on this page.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** The run in which the students propose gives every student their best valid partner and every school its worst.
- ☐ **B.** An instance with n on each side has at most n stable matchings.
- ☐ **C.** A pair who are first on each other's lists appears in every stable matching.
- ☐ **D.** Every stable matching is the output of the algorithm under some order of asks.
- ☐ **E.** If an instance has exactly one stable matching, the two directions agree.
- ☐ **F.** Swapping which side proposes always changes at least one pair.

3.8 *Skill: whether a true list is the best list to submit*

The students are the ones proposing. Can a student end up somewhere they prefer by submitting a list that is not their true ranking?

- ◯ **A.** Yes, by moving a school they are likely to get to the top of the list.
- ◯ **B.** No, and the same holds for the schools.
- ◯ **C.** No. Telling the truth is the best list to hand in for whoever proposes, no matter what anybody else hands in. That last part is what makes it a dominant strategy.
- ◯ **D.** Yes, but only for a student who knows what everybody else submitted.

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.9 *Skill: which assumptions are load-bearing*

Every list on this page is strict and ranks the whole other side. Allow a list to leave people off. What actually breaks?

- ◯ **A.** It finishes stops being true.
- ◯ **B.** Nobody is left out stops being true, and stability has to be restated so that ending with nobody beats being matched to somebody you left off.
- ◯ **C.** It holds stops being true.
- ◯ **D.** The answer starts to depend on the order of the asks.

Why not the others? Pick the one you almost chose and say what is wrong with it.

PART 4 · LEARN

Proof tools and variants

A measure of progress

To show a loop stops, find one number that goes up by at least one every step and cannot pass a ceiling. Here: the number of (asker, asked) pairs where the ask has already happened. +1 each step, never above n^2. "Number of free people" does not work: a bump leaves it unchanged.

"Finishes" and "nobody left out" are two theorems

The loop stops when no free asker has anyone left to ask. The step to prove: a free asker always has somebody left, by counting (if they had asked everyone, all n on the other side are holding someone, so n askers are held, leaving no room for this one).

Claims about *every* stable matching

No algorithm appears. The shape: assume some stable matching is "wrong", count until two people are provably misplaced, show they would both switch. Example: if every list ranks the k good people above the bad ones, a good student with a bad school means one good school has a bad student, and those two block.

Variants

- **Forbidden pairs** (incomplete lists). Not the same as ranking last: a forbidden pair isn't a bad matching, it isn't a matching. People may end unmatched; stability gains cases (a matched person who prefers some unmatched partner; two unmatched people allowed to each other). The loop changes one word: keep asking while a free asker has someone left they are *allowed* to ask. Largest is not the same as stable here: CLRS (3rd edition, §26.3) calls a matching with as many pairs as possible a **maximum matching**, and with forbidden pairs the stable matching can have fewer pairs than that, though never fewer than half.
- **Hospitals and residents.** Hospitals have q posts; more students than posts. Each hospital holds its q best askers so far. A hospital with q posts is q hospitals with one post and the same list, so the old proofs carry over. Stability gains a case: an unmatched student and a hospital that prefers them to someone it took. (The US residency match used this before Gale and Shapley published in 1962.)
- **Ties.** A pair blocks *strongly* if both strictly prefer each other; *weakly* if one strictly prefers and the other is at least indifferent. Break ties arbitrarily and run the process: no strongly blocking pair survives, so that kind always exists. A matching with no weakly blocking pair need not exist.

TRY IT

Two students both rank MIT above NYU; both schools are indifferent between the two students. List both possible matchings and find a weakly blocking pair in each.

PART 4 · QUESTIONS

Tier 4: The chapter's own exercises: variants

4.1 *Skill: choosing the quantity that bounds a loop*

The proof that the process stops needs a quantity that goes up by at least one with every question asked, and cannot pass a ceiling. Which of these is one?

- ◯ **A.** The number of people who are currently free.
- ◯ **B.** The number of pairs currently held.
- ◯ **C.** The number of pairs (asker, asked) such that the asker has already asked that person.
- ◯ **D.** How far down their own list each asker has reached.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.2 *Skill: the claim that is easiest to assume*

```
Claim: when the process stops, every person on both sides is matched.
The lists are strict and complete, and the sides are the same size.
```

Build the proof.

LINE 1 · WHAT IS BEING CLAIMED

- ◯ **A.** That the process stops.
- ◯ **B.** That every asker ends matched, so the result is a perfect matching, the textbook name for one where nobody is left out.
- ◯ **C.** That every asker ends with the best person who did not turn them away.

LINE 2 · THE STEP TO PROVE FIRST

- ◯ **A.** If an asker is free at some point, there is somebody they have not yet asked.
- ◯ **B.** If an asker is free at some point, somebody has turned them away.
- ◯ **C.** Anybody who has been asked is holding somebody from then on.

LINE 3 · THE ARGUMENT FOR IT

- ◯ **A.** Suppose an asker is free and has asked everybody. Then every person on the other side has been asked, so every one of them is holding somebody, so n askers are held. This one is not, and there are only n.
- ◯ **B.** Suppose an asker is free and has asked everybody. Then everybody turned them away, so they are last on every list, which cannot happen to more than one person.
- ◯ **C.** Suppose an asker is free and has asked everybody. Then n^2 questions have been asked and the process has to stop.

LINE 4 · FINISHING

- ◯ **A.** So every asker is matched, and by symmetry so is everybody on the other side.
- ◯ **B.** So the loop can only exit with no free asker, and the held pairs are then a matching covering everybody.
- ◯ **C.** So there are at most n^2 steps and the result covers everybody.

One line per step. The proof counts only if every line is right.

4.3 *Skill: proving something about every stable matching at once*

```
Everybody is either good or bad, and there are k good students and k good schools.
Every list ranks every good person on the other side above every bad one.
```

In every stable matching, every good student is matched to a good school. What is the argument?

- ◯ **A.** Because the process settles the good people first, and by the time it reaches the bad ones the good ones are taken.
- ◯ **B.** Suppose a good student had a bad school. Then at most k − 1 of the k good schools have good students, so some good school has a bad student, and those two would both switch.
- ◯ **C.** Because a good student ranks every good school above every bad one, so they would never accept a bad school.
- ◯ **D.** It is false. With k = 1 the single good student can end up with the single bad school.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.4 *Skill: which part a variant actually changes*

Some pairs are forbidden outright, so a student is ranked only by the schools they could actually attend. What does that change?

- ◯ **A.** Nothing. A forbidden pair is the same as being ranked last.
- ◯ **B.** People can end up matched to nobody, so stability grows three more cases: a matched student who would rather have some unmatched school than the one they have, a matched school who would rather have some unmatched student, and two people who both ended with nobody and are allowed each other.
- ◯ **C.** A stable matching need not exist any more, so the question becomes which instances have one.
- ◯ **D.** The process stops finishing, because an asker can run out of list.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.5 *Skill: telling the largest matching from the stable one*

```
Some pairs are forbidden, so each list names only who that person is allowed.
Students, best first:
  Priya  MIT > NYU
  Sam    MIT
Schools, best first:
  MIT    Priya > Sam
  NYU    Priya
```

CLRS, section 26.3, calls a matching a maximum matching when no other matching has more pairs in it. Which matching of these lists is stable, and is it a maximum one?

- ◯ **A.** Priya with NYU and Sam with MIT. It is the maximum matching, nobody is left out, and a stable matching leaves out as few people as it can.
- ◯ **B.** None. Sam and NYU cannot both be placed without moving Priya, so some pair always wants to switch.
- ◯ **C.** Priya with MIT, and Sam and NYU with nobody. It has one pair, and the maximum matching has two.
- ◯ **D.** Both Priya with MIT on its own and the maximum matching hold, and which one a run ends at depends on who asks.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.6 *Skill: unequal sides and more than one seat*

Hospitals have several posts each, there are more graduating students than posts, and every post has to be filled. How does the algorithm handle it?

- ◯ **A.** It cannot: with unequal numbers no stable assignment exists.
- ◯ **B.** Run the ordinary process on as many students as there are posts, and set the rest aside.
- ◯ **C.** Students ask, and each hospital with q posts holds its q best askers so far instead of just one. Stability gains a case for students who end with nothing, and a stable assignment always exists.
- ◯ **D.** Assign every student to the hospital that ranks them highest, then fix the hospitals that end up over their limit.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.7 *Skill: what a strong instability needs*

```
Students, best first (= means tied):
  Priya          MIT = UMass Amherst > NYU
  Sam            MIT > NYU > UMass Amherst
  Ravi           NYU > MIT = UMass Amherst
Schools, best first:
  MIT            Sam = Ravi > Priya
  UMass Amherst  Priya > Sam > Ravi
  NYU            Priya = Sam = Ravi
The matching: Priya–NYU · Sam–UMass Amherst · Ravi–MIT
```

Some of these lists have ties. Tick every pair that is a strong instability of this matching.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** Priya and MIT
- ☐ **B.** Priya and UMass Amherst
- ☐ **C.** Sam and MIT
- ☐ **D.** Sam and NYU
- ☐ **E.** Ravi and UMass Amherst
- ☐ **F.** Ravi and NYU

4.8 *Skill: what a weak instability needs*

```
Students, best first (= means tied):
  Priya          MIT = UMass Amherst > NYU
  Sam            MIT > NYU > UMass Amherst
  Ravi           NYU > MIT = UMass Amherst
Schools, best first:
  MIT            Sam = Ravi > Priya
  UMass Amherst  Priya > Sam > Ravi
  NYU            Priya = Sam = Ravi
The matching: Priya–NYU · Sam–UMass Amherst · Ravi–MIT
```

Same lists, same matching. Now tick every pair that is a weak instability.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** Priya and MIT
- ☐ **B.** Priya and UMass Amherst
- ☐ **C.** Sam and MIT
- ☐ **D.** Sam and NYU
- ☐ **E.** Ravi and UMass Amherst
- ☐ **F.** Ravi and NYU

4.9 *Skill: checking one matching against both definitions*

```
Students, best first (= means tied):
  Priya          MIT = UMass Amherst > NYU
  Sam            MIT > NYU > UMass Amherst
  Ravi           NYU > MIT = UMass Amherst
Schools, best first:
  MIT            Sam = Ravi > Priya
  UMass Amherst  Priya > Sam > Ravi
  NYU            Priya = Sam = Ravi
The matching: Priya–UMass Amherst · Sam–NYU · Ravi–MIT
```

The same lists, and a different matching. Is it free of strong instabilities? Is it free of weak ones?

- ◯ **A.** Free of both.
- ◯ **B.** Free of strong ones, but not of weak ones: Sam and MIT are a weak instability.
- ◯ **C.** Free of neither: Sam and MIT are a strong instability.
- ◯ **D.** Free of weak ones, but not of strong ones.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.10 *Skill: where indifference counts*

```
Students, best first (= means tied):
  Priya          MIT > UMass Amherst = NYU
  Sam            UMass Amherst = MIT > NYU
  Ravi           MIT > NYU > UMass Amherst
Schools, best first:
  MIT            Sam > Priya = Ravi
  UMass Amherst  Priya = Ravi > Sam
  NYU            Sam > Ravi > Priya
The matching: Priya–UMass Amherst · Sam–NYU · Ravi–MIT
```

New lists. In this matching, what are Priya and MIT?

◯ **A.** A strong instability.

◯ **B.** Neither: MIT would gain nothing, so nothing happens.

◯ **C.** A weak instability and not a strong one.

◯ **D.** Neither, because Priya ranks UMass Amherst and NYU level.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.11 *Skill: checking every matching of one instance*

```
Students, best first (= means tied):
  Priya          MIT > UMass Amherst = NYU
  Sam            UMass Amherst = MIT > NYU
  Ravi           MIT > NYU > UMass Amherst
Schools, best first:
  MIT            Sam > Priya = Ravi
  UMass Amherst  Priya = Ravi > Sam
  NYU            Sam > Ravi > Priya
```

The same lists, and all six of their matchings. Tick every one that has no strong instability.

Tick every one that is true. Partly right counts as wrong.

☐ **A.** Priya–MIT · Sam–UMass Amherst · Ravi–NYU

☐ **B.** Priya–MIT · Sam–NYU · Ravi–UMass Amherst

☐ **C.** Priya–UMass Amherst · Sam–MIT · Ravi–NYU

☐ **D.** Priya–UMass Amherst · Sam–NYU · Ravi–MIT

☐ **E.** Priya–NYU · Sam–MIT · Ravi–UMass Amherst

☐ **F.** Priya–NYU · Sam–UMass Amherst · Ravi–MIT

4.12 *Skill: what indifference costs*

```
Allow ties, so a school can be indifferent between two students.
A pair blocks STRONGLY if each strictly prefers the other to their partner.
A pair blocks WEAKLY if one strictly prefers the other, and the other is indifferent or also strictly
prefers.
```

Which of these is true?

- ◯ **A.** Both always exist, because the ties can be broken arbitrarily first.
- ◯ **B.** Neither need exist, because with ties the process can loop.
- ◯ **C.** A matching with no strongly blocking pair always exists; one with no weakly blocking pair need not.
- ◯ **D.** Both need not exist, and which one you get depends on how the ties are broken.

Why not the others? Pick the one you almost chose and say what is wrong with it.

KEEP THIS PAGE

One-page summary

The process

- Free asker asks the next name on their list.
- Asked side holds the best asker so far; the other is turned away / let go.
- Stop when nobody is free. At most n^2 asks.

Holds (stable)

- Broken by two people who would **both** rather have each other (blocking pair).
- Check: for each student, schools above their seat; would any of those prefer them to who it has?

Why it works

- Finishes: names are spent, never returned.
- Nobody left out: anyone asked holds someone forever; counting.
- Holds: you asked every school above yours; each ended with someone it likes better.

Who asks matters

- Askers: best valid partner. Asked: worst valid partner.
- Order of asks never changes the ending.
- Same seat in both runs ⇒ same seat in every stable matching.
- Exactly one stable matching ⇒ both directions agree.
- A mutual first-choice pair is in every stable matching.
- There may be many stable matchings (exponentially many); runs find only the two extremes.

Statements

- Instance = the inputs only (people + strict complete lists).
- Negate: flip every quantifier, keep order, negate the end.
- Disprove "∀I ∃M": one instance + **every** stable matching of it.

Running time

- n^2 asks × O(1) each, using ranking[school][student] built once in $O(n^2)$.

Incentives

- Honesty is dominant for askers; receivers can gain by lying.

Variants

- Incomplete lists / forbidden pairs: unmatched people; stability grows cases.
- Hospitals with q posts = q copies with one post.
- Ties: strongly-stable-free always exists (break ties); weakly-blocking-free may not.

Proof moves

- Measure of progress: one number, +1 per step, capped.
- "First violation" in extremal proofs.
- Count, then exhibit a blocking pair.

ANSWER KEY

Answer key: runs and activities

Try everything before you look.

Worked example check and W1

Students asking: Priya → **UMass Amherst** · Sam → **MIT** · Ravi → **NYU**.

W1, schools asking:

#	asks	whom	what happens
1	MIT	Sam	Sam was holding nobody, so holds MIT.
2	UMass Amherst	Ravi	Ravi was holding nobody, so holds UMass Amherst.
3	NYU	Priya	Priya was holding nobody, so holds NYU.

Your order of asks may differ from this one. The final seats cannot: see question 1.6.

Priya → **NYU** · Sam → **MIT** · Ravi → **UMass Amherst**. Every school got its first choice and every student is worse off or level: Priya drops from UMass (her 2nd) to NYU (her last), Ravi from NYU (1st) to UMass (last). Sam is at MIT either way. **The answer depends on who asks.**

Check table: Priya has NYU and would rather have MIT or UMass Amherst, but each is holding its own first choice (Sam, Ravi). Ravi has UMass Amherst and would rather have NYU or MIT, and again each has its first choice. Sam has his first choice. Nobody can find a partner who would also switch, so it holds. Both runs hold; they are the two extremes.

Warm-up W2

Students asking: Priya → **MIT** · Sam → **NYU**. Schools asking: Priya → **NYU** · Sam → **MIT**. Each student's first choice is a school whose first choice is the other student, so whoever asks gets everything, in two questions and no bumps. Both arrangements hold.

#	asks	whom	what happens
1	Priya	MIT	MIT was holding nobody, so holds Priya.
2	Sam	NYU	NYU was holding nobody, so holds Sam.

Your order of asks may differ from this one. The final seats cannot: see question 1.6.

Warm-up W3

Priya → **MIT** · Sam → **Berkeley** · Ravi → **UMass Amherst** · Maya → **Northeastern**, both ways. 10 asks each way, lots of turning away and nobody ever bumped. When everybody agrees there is only one arrangement that holds, so who asks cannot matter.

#	asks	whom	what happens
1	Priya	MIT	MIT was holding nobody, so holds Priya.
2	Sam	MIT	Turned away: MIT is holding Priya and likes them better.
3	Sam	Berkeley	Berkeley was holding nobody, so holds Sam.
4	Ravi	MIT	Turned away: MIT is holding Priya and likes them better.
5	Ravi	Berkeley	Turned away: Berkeley is holding Sam and likes them better.
6	Ravi	UMass Amherst	UMass Amherst was holding nobody, so holds Ravi.
7	Maya	MIT	Turned away: MIT is holding Priya and likes them better.
8	Maya	Berkeley	Turned away: Berkeley is holding Sam and likes them better.
9	Maya	UMass Amherst	Turned away: UMass Amherst is holding Ravi and likes them better.
10	Maya	Northeastern	Northeastern was holding nobody, so holds Maya.

Your order of asks may differ from this one. The final seats cannot: see question 1.6.

Warm-up W4

Students asking (10 asks): Priya → **NYU** · Sam → **Berkeley** · Ravi → **UMass Amherst** · Maya → **MIT**. Schools asking (8 asks): Priya → **NYU** · Sam → **Berkeley** · Ravi → **UMass Amherst** · Maya → **MIT**. Identical, even though people were bumped along the way: this set of lists has exactly one arrangement

that holds, so both directions must find it. Asking first is only an advantage when more than one arrangement holds.

#	asks	whom	what happens
1	Priya	MIT	MIT was holding nobody, so holds Priya.
2	Sam	NYU	NYU was holding nobody, so holds Sam.
3	Ravi	NYU	Turned away: NYU is holding Sam and likes them better.
4	Ravi	Berkeley	Berkeley was holding nobody, so holds Ravi.
5	Maya	MIT	MIT likes Maya better than Priya: holds Maya, **lets Priya go**.
6	Priya	Berkeley	Turned away: Berkeley is holding Ravi and likes them better.
7	Priya	NYU	NYU likes Priya better than Sam: holds Priya, **lets Sam go**.
8	Sam	Berkeley	Berkeley likes Sam better than Ravi: holds Sam, **lets Ravi go**.
9	Ravi	MIT	Turned away: MIT is holding Maya and likes them better.
10	Ravi	UMass Amherst	UMass Amherst was holding nobody, so holds Ravi.

Your order of asks may differ from this one. The final seats cannot: see question 1.6.

Challenge C1 · the cascade

#	asks	whom	what happens
1	Priya	UMass Amherst	UMass Amherst was holding nobody, so holds Priya.
2	Sam	Michigan	Michigan was holding nobody, so holds Sam.
3	Ravi	MIT	MIT was holding nobody, so holds Ravi.
4	Maya	Berkeley	Berkeley was holding nobody, so holds Maya.
5	Diego	MIT	MIT likes Diego better than Ravi: holds Diego, **lets Ravi go**.
6	Ravi	UMass Amherst	UMass Amherst likes Ravi better than Priya: holds Ravi, **lets Priya go**.
7	Priya	Michigan	Michigan likes Priya better than Sam: holds Priya, **lets Sam go**.
8	Sam	UMass Amherst	UMass Amherst likes Sam better than Ravi: holds Sam, **lets Ravi go**.
9	Ravi	Berkeley	Berkeley likes Ravi better than Maya: holds Ravi, **lets Maya go**.
10	Maya	MIT	MIT likes Maya better than Diego: holds Maya, **lets Diego go**.
11	Diego	Berkeley	Berkeley likes Diego better than Ravi: holds Diego, **lets Ravi go**.
12	Ravi	NYU	NYU was holding nobody, so holds Ravi.

Your order of asks may differ from this one. The final seats cannot: see question 1.6.

Students asking: Priya → **Michigan** · Sam → **UMass Amherst** · Ravi → **NYU** · Maya → **MIT** · Diego → **Berkeley** (12 asks, 7 people let go, all in one unbroken chain set off when Diego asks MIT). Schools asking: Priya → **NYU** · Sam → **Berkeley** · Ravi → **Michigan** · Maya → **MIT** · Diego → **UMass Amherst**, five asks and no bumps. This instance has 4 arrangements that hold:

- Priya → **NYU** · Sam → **UMass Amherst** · Ravi → **Michigan** · Maya → **MIT** · Diego → **Berkeley**
- Priya → **Michigan** · Sam → **UMass Amherst** · Ravi → **NYU** · Maya → **MIT** · Diego → **Berkeley**
- Priya → **Michigan** · Sam → **Berkeley** · Ravi → **NYU** · Maya → **MIT** · Diego → **UMass Amherst**
- Priya → **NYU** · Sam → **Berkeley** · Ravi → **Michigan** · Maya → **MIT** · Diego → **UMass Amherst**

Two of them are reachable by neither run: the process only ever finds the two extremes.

Challenge C2 · good people and bad people

All 4 arrangements that hold:

- Priya → **UMass Amherst** · Sam → **MIT** · Ravi → **Berkeley** · Maya → **NYU**
- Priya → **UMass Amherst** · Sam → **MIT** · Ravi → **NYU** · Maya → **Berkeley**
- Priya → **MIT** · Sam → **UMass Amherst** · Ravi → **Berkeley** · Maya → **NYU**
- Priya → **MIT** · Sam → **UMass Amherst** · Ravi → **NYU** · Maya → **Berkeley**

Priya and Sam (the good students) are always at MIT or UMass Amherst (the good schools). The argument is question 4.3.

Part 2 try-it

2. There is an instance in which every stable matching contains no pair (c, s) that are first on each other's lists. 3. Matchings {(c1,s1),(c2,s2)} and {(c1,s2), (c2,s1)}. Both are stable: in the first both colleges have their first choice, in the second both students do. Neither contains a mutual first pair, because none exists in this instance: c1 tops s2's list but c1 wants s1; c2 tops s1's list but c2 wants s2. That is the full disproof (question 2.6).

Part 3 ranking array

	Priya	Sam	Ravi	Maya
MIT	1	2	3	4
UMass Amherst	2	1	4	3
NYU	1	3	2	4
Berkeley	2	1	3	4

UMass Amherst: ranking[Maya] = 3 < ranking[Ravi] = 4, so it holds Maya and lets Ravi go.

Part 4 try-it

Matching 1: student A at MIT, B at NYU. B strictly prefers MIT, and MIT is indifferent between A and B, so (B, MIT) blocks weakly. Matching 2 is the mirror, blocked weakly by (A, MIT). Every matching has a weakly blocking pair, so none is weakly stable, yet neither has a strongly blocking pair.

ANSWER KEY

Answer key: questions

1.1

Priya → **Berkeley** · Sam → **MIT** · Ravi → **UMass Amherst** · Maya → **NYU**

- **Priya**: Berkeley is her first choice, and the one person Berkeley likes better never asks it. The first question she asks is the last one she needs.
- **Sam**: MIT is his first choice, and MIT would take him over anybody except Priya, who is not asking MIT.
- **Ravi**: MIT turns him away: it is holding Sam and likes Sam better. UMass Amherst is next on his list and has nobody.
- **Maya**: Berkeley turns her away: she is last on its list and it is holding Priya. NYU is next and has nobody.

#	asks	whom	what happens
1	Priya	Berkeley	Berkeley was holding nobody, so holds Priya.
2	Sam	MIT	MIT was holding nobody, so holds Sam.
3	Ravi	MIT	Turned away: MIT is holding Sam and likes them better.
4	Ravi	UMass Amherst	UMass Amherst was holding nobody, so holds Ravi.
5	Maya	Berkeley	Turned away: Berkeley is holding Priya and likes them better.
6	Maya	NYU	NYU was holding nobody, so holds Maya.

Your order of asks may differ from this one. The final seats cannot: see question 1.6.

Takeaway. Nobody is displaced in this run. Two people are turned away on the spot, which is a different thing: being turned away costs you one name off your own list and nothing else, while pushing somebody out sends them back to spend more of theirs.

1.2

Priya → **Berkeley** · Sam → **MIT** · Ravi → **NYU** · Maya → **UMass Amherst**

- **Priya**: Four schools ask her and she trades up her own list every time: she holds MIT, then lets it go for NYU, then lets NYU go for UMass Amherst, then lets that go for Berkeley, which is where she wanted to be all along.
- **Sam**: UMass Amherst asks first and he holds it, then MIT asks and he lets UMass go. Berkeley asks after that and he turns it down, because MIT is his first choice and he already has it. MIT is where the other run left him too.
- **Ravi**: NYU asks him once Priya is gone. Nobody else asks him at all, so he has no say in it beyond yes.
- **Maya**: UMass Amherst asks her last, after Sam and Priya are both out of reach. She is nowhere near the top of anybody's list here.

#	asks	whom	what happens
1	MIT	Priya	Priya was holding nobody, so holds MIT.
2	UMass Amherst	Sam	Sam was holding nobody, so holds UMass Amherst.
3	NYU	Priya	Priya likes NYU better than MIT: holds NYU, **lets MIT go**.
4	MIT	Sam	Sam likes MIT better than UMass Amherst: holds MIT, **lets UMass Amherst go**.
5	UMass Amherst	Priya	Priya likes UMass Amherst better than NYU: holds UMass Amherst, **lets NYU go**.
6	NYU	Ravi	Ravi was holding nobody, so holds NYU.
7	Berkeley	Sam	Turned away: Sam is holding MIT and likes them better.
8	Berkeley	Priya	Priya likes Berkeley better than UMass Amherst: holds Berkeley, **lets UMass Amherst go**.
9	UMass Amherst	Maya	Maya was holding nobody, so holds UMass Amherst.

Your order of asks may differ from this one. The final seats cannot: see question 1.6.

Takeaway. Priya and Sam are in the same seats as before. Ravi and Maya have swapped. Who does the asking changes part of the answer and not all of it, and the part it cannot change is the part that was never in dispute.

1.3

Hold: **A, C**

- **A** ✓ Holds: no student has a school above their seat that would also rather have them. *This is the one the students asking produced.*

- **B** × Broken. Sam and MIT would both switch: Sam has UMass Amherst but ranks MIT higher, and MIT has Ravi but ranks Sam higher. (2 such pairs in all: Sam & MIT, Sam & Berkeley.) *Sam and Ravi have swapped seats from the students-asking run, and nobody else has moved.*
- **C** ✓ Holds: no student has a school above their seat that would also rather have them. *This is the one the schools asking produced, and the answer to "find another one".*
- **D** × Broken. Priya and UMass Amherst would both switch: Priya has MIT but ranks UMass Amherst higher, and UMass Amherst has Ravi but ranks Priya higher. (2 such pairs in all: Priya & UMass Amherst, Priya & NYU.) *Two people who were both settled have swapped, which is the usual way to produce something that looks reasonable and is not.*
- **E** × Broken. Priya and Berkeley would both switch: Priya has NYU but ranks Berkeley higher, and Berkeley has Maya but ranks Priya higher. (2 such pairs in all: Priya & Berkeley, Priya & UMass Amherst.) *Two of the four are at their first choice and nobody is at their last. Comfort is not the test.*
- **F** × Broken. Priya and Berkeley would both switch: Priya has UMass Amherst but ranks Berkeley higher, and Berkeley has Ravi but ranks Priya higher. (6 such pairs in all: Priya & Berkeley, Sam & MIT, Sam & Berkeley, Sam & UMass Amherst, Ravi & MIT, Ravi & NYU.) *Everybody moved, and the further an arrangement gets from either run the more ways there are to break it.*

Takeaway. Two of the six hold, and they are exactly the two the process produced from the two directions. That is not a coincidence here, and it is not a rule either: an instance can have arrangements that hold which neither direction ever reaches.

1.4

Answer: **A**

- **A** ✓ The two runs are the best and the worst any arrangement can do for a given side, so they bracket the rest. Anybody sitting in the same seat at both ends of that range has nowhere else to be. That the two runs really are the two ends is proved later, in the textbook tier.
- **B** × The order changes nothing. Ask the questions in any order at all, with the same side asking, and the ending is the same. That is a property of the process, not of this instance.
- **C** × There is no ranking of schools here beyond the lists on the page, and nothing settles first. Everything stays provisional until the whole thing stops.
- **D** × It has two, and the second run found the other one. They differ on Ravi and Maya, which is precisely why Priya and Sam standing still is worth noticing.

Takeaway. The two runs are the far ends of what is possible. Whoever does not move between them cannot move at all, and whoever does is exactly who the choice of side is deciding for.

1.5

Answer: **C**

- **A** × Writing it out is what exposes the problem: the seats would be the same four seats as the answer above it. The method is sound, and here it was applied to the run it came from.
- **B** × It usually does. On these lists the two runs differ on Ravi and Maya. One of the sets of lists on this page does give the same answer either way, and it does that because there is only one arrangement to be found.
- **C** ✓ Flipping the side is the right instinct and it was pointed at the wrong starting point. From a students-asking run, the second arrangement comes from the schools asking.
- **D** × The process produces one per direction, so two candidates come for free, and on these lists those two are the only two there are. Where there are more, the rest do have to be found by hand.

Takeaway. Both runs are worth doing and worth labelling as you go. The commonest way to answer "find another one" wrongly is to hand back the one already on the page.

1.6

Answer: **D**

- **A** × A worksheet fixes an order so that two people can compare their working line by line, not because the ending depends on it.
- **B** × Displacements are where two routes differ most, and the endings still agree. In the schools-asking run here Priya holds four different schools before she settles, letting three of them go, and where she ends does not depend on which school asked her first.
- **C** × The ending is fixed per direction even when several arrangements hold. These lists give two, and the students asking always produce the same one of them.
- **D** ✓ Whoever is asking ends up with the best seat available to them in any arrangement that holds, and that is a fact about the lists rather than about the route. Nothing about the ending is left to the choices made along the way, so there is nothing for two people to disagree about.

Takeaway. The route is not part of the answer. That is worth knowing before a problem set, because the working shown in a solution will rarely be the working you did.

1.7

Priya → **NYU** · Sam → **MIT** · Ravi → **UMass Amherst**

- **Priya**: MIT asks her first and she holds it. Then NYU asks, her first choice, and she lets MIT go. UMass Amherst asks later and she turns it away: she already has NYU.
- **Sam**: UMass Amherst asks him first. Then MIT asks, which he likes better, so he lets UMass go and stays at MIT.
- **Ravi**: UMass Amherst gets to him last, after Sam has let it go and Priya has turned it away. Nobody else ever asks him, so UMass Amherst it is.

#	asks	whom	what happens
1	MIT	Priya	Priya was holding nobody, so holds MIT.
2	UMass Amherst	Sam	Sam was holding nobody, so holds UMass Amherst.
3	NYU	Priya	Priya likes NYU better than MIT: holds NYU, **lets MIT go**.
4	MIT	Sam	Sam likes MIT better than UMass Amherst: holds MIT, **lets UMass Amherst go**.
5	UMass Amherst	Priya	Turned away: Priya is holding NYU and likes them better.
6	UMass Amherst	Ravi	Ravi was holding nobody, so holds UMass Amherst.

Your order of asks may differ from this one. The final seats cannot: see question 1.6.

Takeaway. Six questions, the same six the slides go through: a asks 1, b asks 2, c asks 1 and bumps a, a asks 2 and bumps b, b is turned away by 1, b asks 3. Only one arrangement holds for these lists, so the students asking would land in exactly the same seats.

1.8

Hold: **B, D**

- **A** × Broken. Priya and UMass Amherst would both switch: Priya has MIT but ranks UMass Amherst higher, and UMass Amherst has Ravi but ranks Priya higher. (2 such pairs in all: Priya & UMass Amherst, Sam & UMass Amherst.) *This is the seating on the slide, the one the clicker asks you to break. The pair it is after is b and 1.*
- **B** ✓ Holds: no student has a school above their seat that would also rather have them. *This is where the schools asking ends up: every school but NYU gets its first choice.*
- **C** × Broken. Priya and UMass Amherst would both switch: Priya has NYU but ranks UMass Amherst higher, and UMass Amherst has Ravi but ranks Priya higher. (2 such pairs in all: Priya & UMass Amherst, Priya & MIT.) *Sam has his first choice, which is not the same as nobody being able to break it.*
- **D** ✓ Holds: no student has a school above their seat that would also rather have them. *This is where the students asking ends up: Priya and Sam both have their first choice.*

Takeaway. On the slide the answer is (b, 1): UMass Amherst holds Ravi but would rather have Priya, and Priya holds MIT but would rather have UMass Amherst. Two people, both better off, is all it takes. The two seatings that hold are the two the process reaches, one from each side.

1.9

Priya → **MIT** · Sam → **UMass Amherst** · Ravi → **NYU**

- **Priya**: MIT asks her first and nobody else ever asks her, so she stays at MIT, which is second on her own list.
- **Sam**: UMass Amherst asks him first and nobody else does, so he stays at UMass Amherst, his second choice.
- **Ravi**: NYU asks him and he wants NYU most, so NYU is where he stays.

#	asks	whom	what happens
1	MIT	Priya	Priya was holding nobody, so holds MIT.
2	UMass Amherst	Sam	Sam was holding nobody, so holds UMass Amherst.
3	NYU	Ravi	Ravi was holding nobody, so holds NYU.

Your order of asks may differ from this one. The final seats cannot: see question 1.6.

Takeaway. Three questions and done: every school's first choice is free when it asks. With the students asking it is also three questions, and Priya and Sam swap, each getting their first choice. Ravi and NYU want each other most, so they are together whoever asks.

2.1

Answer: **D**

- **A** × It is stable: each college has its first choice, so no college would move, and a blocking pair needs both sides to want it. But it is not the only one.
- **B** × It is stable: each student has their first choice, so no student would move. But it is not the only one.
- **C** × A stable matching always exists, since the process always produces one. Here it produces two different ones depending on which side proposes.
- **D** ✓ The colleges proposing gives the first, where every college has its first choice. The students proposing gives the second, where every student has theirs. Neither has a blocking pair.

Takeaway. Stable matchings are not unique. The college-proposing run gives the colleges their best stable partners and the students their worst; the student-proposing run does the reverse. Which side proposes chooses between them.

2.2

Answer: **B**

- **A** × Those are outputs and a choice about how to run the algorithm. An instance is fixed before any of that: the same instance can be run in either direction, which is exactly what the first tier was doing.
- **B** ✓ It is the question, written out in full. Nothing about how it will be solved is part of it, and nothing about the answer is either.
- **C** × An instance exists whether or not anybody runs anything on it. The algorithm is one way of answering the question; the instance is the question.
- **D** × One matching is a candidate answer to an instance. An instance of size n has n! of them, and the statement is about which of those are stable.

Takeaway. A claim that starts "in every instance" is quantifying over inputs, so pinning down what an input is settles what is being claimed before any of the logic starts. Two more things to carry across from tier 1. The textbook says colleges where this page says schools, and writes c for a college and s for a student: same two sides, shorter names. And a matching is stable when it has no blocking pair, which is two people who would both rather have each other under its textbook name.

2.3

Answer: **B**

- **A** × This asks every pair in M to be a mutual first choice, which is the claim that everybody gets their first choice. The English says "containing a pair", and a matching that contains one may contain nothing else of the kind.
- **B** ✓ Three English phrases, three quantifiers, in the order they are written: "in every instance" is ∀I, "there is a stable matching" is ∃M, and "containing a pair" is ∃(c, s).
- **C** × This asks every stable matching to contain such a pair. The English says "there is a stable matching", which is ∃M. The gap between these two is exactly what the flawed answer two questions along falls into.
- **D** × This says some instance has such a matching, which is much weaker and easy to satisfy: any instance where two people are each other's first choice will do. "In every instance" is ∀I, and it is the first thing in the sentence.

Takeaway. Underline the English phrases that act as quantifiers and the formula writes itself, in the order they appear. Every dispute about a statement like this one is a dispute about those three phrases.

2.4

True: **A, C** · False: B, D, E, F

- **A** ✓ The negation one line before it is finished: the outer two quantifiers have turned over and the ¬ is still sitting where the last ∃ used to be. Correct as it stands, and one push away from the finished form, ∃I ∀M ∀(c, s) ∈ M : ¬φ.
- **B** × Only the inside was negated. This says every instance has a stable matching in which no pair is a mutual first choice, which can be true at the same time as the original, so it cannot be its negation.
- **C** ✓ The finished form, and the one to aim for: some instance, every stable matching of it, every pair in that matching. Pushing the ¬ through the last ∃ in ∃I ∀M ¬(∃(c, s) ∈ M : φ) turns it into this.
- **D** × The ∃M should have become ∀M. As written this only produces one bad matching, and the original claim asks for one good one, so both can hold together.
- **E** × The innermost quantifier was left as ∃. This says every stable matching has some pair that is not a mutual first choice, which nearly every matching satisfies, including ones that also contain a mutual first pair.
- **F** × Too strong by a long way: it claims no instance anywhere has such a pair, and plenty do. Negating "for every instance" gives "for some instance", never "for no instance".

Takeaway. Negation walks left to right: every quantifier turns into the other kind, nothing moves, and what is left at the end is a ¬ that can be pushed in or left standing. Two of these are the same statement at two points in that walk.

2.5

Answer: **C**

- **A** × "In every instance" survived unnegated. This says the statement fails everywhere, and the negation only needs it to fail once.
- **B** × Only the innermost part was negated and ∀I ∃M was left standing. This and the original can both be true of the same instance, which is the sign that one is not the negation of the other.
- **C** ✓ "In every instance" became "there is an instance", "there is a stable matching" became "every stable matching", and the relation at the end went negative. Three phrases, three changes.
- **D** × "Some stable matching lacks one" where the negation needs "every stable matching lacks one". This is the flawed answer in the next question, written out in English.

Takeaway. The English phrases carrying the quantifiers are "in every instance", "there is a stable matching" and "containing a pair". Negating the sentence means turning over each of the three, and nothing else in it moves.

2.6

Answer: **A**

- **A** ✓ This is the ∃M that became ∀M under negation, and it is the whole of the work. The other stable matching here is {(c1, s2), (c2, s1)}, in which both students do get their first choice, and it has no mutual first pair either. Saying so finishes the proof.
- **B** × M is stable. c1 has s1 and c2 has s2, which are their first choices, so neither college would move anywhere, and it takes two. Calling a stable matching unstable is the other way to lose this question.
- **C** × The claim is about a pair that is first on both lists, so it is about both sides at once. Noticing that no student is matched to their own top college does settle it for this M, since a pair needs first(c, s) as well.
- **D** × The negation asks for an instance in which every stable matching lacks such a pair. One matching is an example, and an example is not the universal claim the negation makes about that instance.

Takeaway. This instance is on the page as "Two who want what wants somebody else". Nobody in it is first on the list of anybody who is first on theirs, which is why both of its arrangements lack the pair. The argument had the right instance and stopped one matching short of showing it.

2.7

Lines: **1A · 2C · 3A · 4B · 5A**

LINE 1 · WHAT HAS TO BE PRODUCED

- **A** ✓ This is the negation, read off the formula: ∃I, then ∀M. The instance is the easy half and the ∀M is the half that gets skipped.
- **B** × That is one example, and the negation makes a universal claim about the instance: every stable matching of it, not some.
- **C** × Far stronger than the negation, and false. Any instance where two people are each other's first choice has one. Negating "for every instance" gives "for some instance".

LINE 2 · THE INSTANCE

- **A** × Here c1 and s1 are each other's first choice, so the matching pairing them is stable and contains exactly the pair the statement asks for. This instance confirms the statement instead of breaking it.
- **B** × With one of each there is a single matching, and that pair is first on both lists by default. The smallest instance that can fail is two and two.
- **C** ✓ Arranged in a cycle: c1 wants s1, s1 wants c2, c2 wants s2, and s2 wants c1. Each college has a top student whose own top choice is the other college, so no pair anywhere is first on both lists. That is stronger than the proof strictly needs, and it is what makes the last step one sentence.

LINE 3 · WHAT TO CHECK

- **A** ✓ With two and two there are only two matchings at all, and both turn out to be stable, so the whole check is two lines. Saying that both are stable is part of the proof rather than an aside.
- **B** × It returns one stable matching per direction. Two runs is two matchings, and there is no general reason those are all of them. Here they happen to be, and that is something to state, not to assume.
- **C** × More work than the claim needs. The statement only ever talks about stable matchings, so the unstable ones are outside it.

LINE 4 · THE CHECK

- **A** × True of the first matching and false of the second, where both students do get their first choice. The second is still fine, for the other reason: neither student is first on the list of the college they got.
- **B** ✓ One sentence covers both matchings, because it is a fact about the instance rather than about either of them. Any matching of this instance lacks the pair, stable or not.
- **C** × The second is stable. Both students have their first choice in it, so neither would move, and it takes two.

LINE 5 · THE CONCLUSION

- **A** ✓ One instance, a universal claim about it, and the negation is met. Nothing larger was claimed and nothing larger was needed.
- **B** × The first half overshoots into a claim about every instance, which is false and was never shown. One instance is all the negation asks for.
- **C** × The statement was never about what the algorithm finds. It says some stable matching contains such a pair, however anybody arrives at it.

Takeaway. A disproof of "for every instance" is one instance plus a universal claim about it. Finding the instance is the half people do; ruling out every stable matching of it is the half that makes it a proof.

3.1

Answer: **B**

- **A** × Anybody let go asks again. In the schools-asking run on the four-by-four instance, UMass Amherst asks three times on its own.
- **B** ✓ Each ask uses up one name from one list, and a used name is never put back. With n lists of n names there are n^2 names in total, so there are at most n^2 asks, and that bound is fixed before the run starts.
- **C** × The process never enumerates matchings, it walks down lists. How many candidate answers exist has nothing to do with how much work is done.
- **D** × A chain of displacements is long, not exponential: the cascade preset moves seven people from one question. Every displacement still spends a name off a list, so the same n^2 ceiling holds.

Takeaway. This is the running-time argument in one line, and it is a counting argument rather than a reading of the code: no name is ever returned to a list, so n lists of n names cap the whole run. The exact maximum is a little under the bound, n squared minus n plus 1, and the saving belongs to a receiver rather than to a proposer: every receiver is asked at least once, and the last one to be asked at all is asked exactly once rather than up to n times. Nothing downstream cares about the difference, which is rather the point of a bound.

3.2

Answer: **C**

- **A** × Three would be every school asking once and being held. Here MIT is let go by Priya, and UMass Amherst is let go by Sam and then turned away by Priya, and each of those costs one more ask.
- **B** × Four is the count with the students asking. Same lists, other side, a different count.
- **C** ✓ Three first asks, and one more for each time a school is let go or turned away: MIT once, UMass Amherst twice.
- **D** × Nine is n^2, the ceiling the counting argument gives for three a side. The ceiling is not the count.

Takeaway. A count is only as good as the trace behind it. Guessing from n, or from the ceiling, or from the other side's run, gives a different number; writing down every ask and what it ended in gives this one.

3.3

Answer: **D**

- **A** × Four would be every school held on its first ask. Priya alone is asked by all four schools and lets three of them go.
- **B** × Eight counts the four first asks and the four let-gos, and misses the one ask that was turned away on the spot.
- **C** × Sixteen is n^2 for four a side, the ceiling. UMass Amherst asks more than anybody here, and it asks three times, not four.
- **D** ✓ Four first asks, four more after somebody is let go, and one more after Sam turns Berkeley away.

Takeaway. Counting a run is bookkeeping, and it rewards the same habit as working it: one line per ask, and a note of what each one ended in. The total is then the number of lines.

3.4

Answer: **B**

- **A** × The ending is the same because only one matching holds. The route there is not, and the count is a count of the route.
- **B** ✓ With the students asking, Priya is let go by MIT, Sam by NYU and Ravi by Berkeley, and three asks are turned away: four first asks and six more. With the schools asking there are two let-gos and two turn-aways: four and four.
- **C** × The right two numbers the wrong way round. In the schools-asking run only UMass Amherst and Berkeley ever ask more than once.
- **D** × Sixteen is n^2, the ceiling. Neither run reaches it: nobody on either side asks all four names on their list except UMass Amherst when the schools ask, and Ravi when the students ask.

Takeaway. How many asks a run makes is a fact about the lists and about which side asks, not about n alone, and not about where the run ends. The workbench on this page counts them for any lists you write.

3.5

Answer: **C**

- **A** × The lists arrive in order: being in order is what makes them preference lists. Sorting changes nothing, because the cost is in answering "where does this name sit", not in the order of the names.

- **B** × Worth doing, and not this. Choosing who asks next is already constant time from a stack or a queue. The cubic term comes from the comparison inside the ask.
- **C** ✓ Building all n of those arrays costs $O(n^2)$, which is already the budget, and every comparison afterwards is two array reads rather than a scan. "Do I prefer this one to the one I am holding" becomes a comparison of two numbers.
- **D** × It is O(n) only if the answer has to be searched for. The ranking is fixed for the whole run, so it can be inverted once and looked up forever after.

Takeaway. The procedure and its running time are two separate claims, and the second one is as much about how the input is stored as about what the code does. This is the usual shape of an $O(n^2)$ bound: a count of the operations, plus a data structure that makes each one constant.

3.6

Lines: **1A · 2B · 3A · 4C · 5A**

LINE 1 · THE SETUP

- **A** ✓ Everything rests on "first". It is what lets the next step say that nobody has yet been rejected by a valid partner, which is the only fact the argument has to work with.
- **B** × Without "first" the next step has nothing: other rejections by valid partners may already have happened, and the argument needs to rule exactly those out.
- **C** × The last one is no easier to reason about than an arbitrary one. The leverage is in there being no earlier violation, not in there being no later one.

LINE 2 · WHAT THE REJECTION GIVES YOU

- **A** × That is why p was asking r at all, and it is not what the rejection tells you. What the rejection tells you is about r, not about p.
- **B** ✓ A rejection only ever happens because the receiver holds somebody it likes better, and the definition of valid partner hands over the matching M. Both objects are now on the table.
- **C** × Nothing in the process consults how the proposer ranks the receiver at the moment of a rejection. The receiver decides on its own list alone.

LINE 3 · WHAT Q PREFERS

- **A** ✓ This is where "first" is spent. q is at r, so everything above r on their list has already rejected them, and none of those can have been a valid partner, because this is the first time in the whole run that a valid partner rejects anybody. So every valid partner of q is r or below it, and their partner in M is one of those.
- **B** × The process improves the receiver's position over time, never the proposer's: a proposer moves down their own list and never back up. This has the direction of the argument backwards.
- **C** × Stable matchings do not give everybody their first choice. On the very first instance on this page, nobody gets everything they want and the arrangement still holds.

LINE 4 · THE CONTRADICTION

- **A** × p and r are matched to each other in M, and a pair that is already together cannot block. The blocking pair is the other one.
- **B** × M pairs q with whoever it pairs them with; nothing said it pairs q with r. The contradiction is about stability, not about M contradicting itself.
- **C** ✓ Both halves of a blocking pair, each established by one of the two previous steps. M was chosen to be stable, so the assumption that started the proof is what has to go.

LINE 5 · WHAT FOLLOWS

- **A** ✓ A proposer walks down their list and stops at the first receiver that keeps them. If no valid partner ever rejects them, the one they stop at is the best valid partner on the list, whatever route the run took to get there.
- **B** × Only if their first choice is a valid partner. The claim is that nobody does better than this run in any stable matching, not that everybody gets what they wanted.
- **C** × The two sides are not on a common scale, so there is nothing to compare. The statement is that each proposer does as well as that proposer can do in any stable matching.

Takeaway. The receiving side gets the mirror image, and the proof is the same one read backwards: every receiver ends with the worst valid partner it has. The two halves have names, proposer-optimal and receiver-pessimal, and they are one theorem rather than two. That is the asymmetry the first tier was looking at, stated as a fact about every instance instead of as two runs that came out differently.

3.7

True: **A, C, E** · False: B, D, F

- **A** ✓ The first half is the theorem the previous question proves, and the second half is its mirror: best for every student is worst for every school, at the same time. On the four-by-four instance the students-asking run leaves NYU with Maya and UMass Amherst with Ravi, and the other arrangement gives each of them the one it preferred.
- **B** × The number can be exponential in n. Instances are known with roughly 2^(n/2) of them, which is why "find another one" is a different problem from "find one".

- **C** ✓ Any matching that separates them is blocked by them: each would rather have the other than anybody, so both would switch. This is the converse of the tier 2 counterexample, which worked precisely because no such pair existed there.
- **D** × The order of asks changes nothing, so one side proposing produces exactly one matching and the two directions produce at most two. The cascade instance on this page has four stable matchings, and two of them are reachable by neither run.
- **E** ✓ Both runs return stable matchings, and there is only one to return. One of the presets on this page is built around exactly this, so that the asymmetry does not get mistaken for a law.
- **F** × Only when more than one stable matching exists. When there is exactly one, the two runs return the same matching, and even when there are several, the people matched the same way in both extremes are matched that way in all of them.

Takeaway. The two runs are the two far ends of the whole set of stable matchings: whichever side asks gets its best end, and the other side gets its worst end at the same moment. They are not two samples from the middle. Everything in between is invisible to the algorithm, and the questions a problem set asks are usually about that in-between.

3.8

Answer: **C**

- **A** × Where a school sits on your own list only decides when you ask it. You are held or rejected on the school's ranking, never on how eager you looked, so promoting a school makes you ask it earlier and be rejected earlier.
- **B** × The schools are on the receiving side, and they can gain. A school that truncates its list, refusing anybody below a cut, can end with a student it prefers. Real matching markets worry about the receiving side for exactly this reason.
- **C** ✓ A theorem in its own right, due to Dubins and Freedman and independently to Roth, and it needs more than the previous question. A lie changes the whole problem: the run comes out stable for the lists that were handed in, not for the true ones, so getting from there back to a claim about what people really wanted has to be argued rather than quoted. What it buys is worth the work, because it holds whatever anybody else does.
- **D** × Knowing the other lists buys a proposer nothing here. The guarantee does not depend on what anybody else does: no other list beats the true one, whatever the others turn out to be.

Takeaway. Telling the truth is the best move for whoever asks and not for whoever is asked, and no process that always ends stable can make it the best move for both sides at once. That is a theorem, not a gap in this particular algorithm.

3.9

Answer: **B**

- **A** × It still finishes, and for the same reason: every ask spends a name off a list, and the lists are now shorter rather than longer.
- **B** ✓ The counting argument that nobody runs out of list assumed a list with every name on it. Once lists are short, people can end unmatched, and a pair only blocks if each prefers the other to what they have, where "nobody" is now a possible thing to have.
- **C** × The result is still stable under the restated definition. What changes is that some people end with nobody, not that two of them would both rather switch.
- **D** × It does not. The proposing side still ends with its best valid partner, and which people end up unmatched is the same in every stable matching, which is a theorem in its own right.

Takeaway. Ties are the harder relaxation, and the next tier takes them seriously: one notion of a blocking pair splits into two, and only one of the two can always be avoided. Strictness is not a convenience, it is what makes the single clean answer exist.

4.1

Answer: **C**

- **A** × It can stay where it is from one step to the next: a displacement frees the person let go and settles the person asking, leaving the count exactly where it was, and a turn-away changes nothing at all. A quantity that is allowed to stand still bounds nothing. It never rises either, which is worth noticing and does not rescue it.
- **B** × A displacement swaps one held pair for another and leaves it exactly where it was, and so does a turn-away. Two of the three things that can happen do not move it.
- **C** ✓ Every step adds exactly one, because nobody asks the same person twice, and it can never exceed n^2. So there are at most n^2 steps. That is the entire proof, and it is a counting argument rather than anything about what the steps do.
- **D** × This is n numbers rather than one, and it is the right idea half-finished: add them up and you have exactly the count of pairs already asked, since how far the askers have collectively reached is how many questions have been asked. Left as a list of n numbers it bounds nothing, because no single one of them has to move when a question is asked.

Takeaway. Finding a measure of progress is the standard way to bound a loop, and all the art is in the choice. Two of the quantities here are allowed to stand still, which disqualifies them, and one is the right idea left as n separate numbers instead of added into one.

4.2

Lines: **1B · 2A · 3A · 4B**

LINE 1 · WHAT IS BEING CLAIMED

- **A** × A separate theorem with a separate proof, and this one needs it: stopping and stopping in a good state are different claims, and a process could perfectly well stop with somebody left over.
- **B** ✓ The whole claim, and what makes it worth proving is that the loop exits when no asker is free, which does not by itself say that nobody ran out of list.
- **C** × They end with the last person they asked, which is the worst of those who did not turn them away. True or false, it is not what perfect means, which is only that nobody is left out.

LINE 2 · THE STEP TO PROVE FIRST

- **A** ✓ This is the load-bearing step. With it, the loop can only exit for the right reason, since the exit condition is that no free asker has anybody left to ask.
- **B** × True once they have asked anybody at all, and no use here. What is needed is that they still have somebody left, not that somebody has already refused.
- **C** × True, and it is the ingredient rather than the step: it gets used inside the proof of the step above. On its own it says nothing about the askers.

LINE 3 · THE ARGUMENT FOR IT

- **A** ✓ Counting, and nothing else. The held pairs form a matching, so n people held means n distinct askers held, which uses up everybody and leaves no room for the free one.
- **B** × Being turned away puts you below whoever was being held at that moment, not last. And being last on every list is perfectly possible for one person anyway.
- **C** × That is the other theorem arriving early. Running out of steps says the process ends; it says nothing about who is matched when it does.

LINE 4 · FINISHING

- **A** × Symmetry is not available here: the two sides do not play the same role, and the whole page is about how differently they come out. What settles the other side is counting, since the held pairs are a matching between equal-sized sides.
- **B** ✓ The exit condition is that no free asker has anybody left to ask. The step just proved says a free asker always does have somebody left, so the loop can only exit with nobody free.
- **C** × The step count is true and is not what was just shown. Two theorems that get run together is exactly the mistake this question exists for.

Takeaway. It finishes and nobody is left out are two theorems, not one, and the second is the one people assume. What carries it is a counting argument in the middle: everybody asked is holding somebody, and there are only so many people to hold.

4.3

Answer: **B**

- **A** × The process has no notion of good and bad, and a good person can be displaced very late. The claim is about every stable matching, including ones no run ever produces, so no argument about the order of asks can reach it.
- **B** ✓ Counting, then one blocking pair. The misplaced good student uses up a good school without occupying it, so some good school is left with a bad student, and that good student and that good school each prefer the other to what they have.
- **C** × A matching is just a list of who is with whom, with nobody asking and nobody answering. The process has accepting and turning away in it; an arrangement on its own does not, and stability is the only test it has to pass. People end up low on their own list all the time when nobody better wants them.
- **D** × Try it. If the one good student had a bad school, the one good school would have a bad student, and those two are each other's first choice. The pair blocks, so no such matching is stable.

Takeaway. This is the shape of most claims about every stable matching: assume one is wrong, count until two people are provably misplaced, and show they block. No algorithm appears anywhere in it.

4.4

Answer: **B**

- **A** × Last is still acceptable. A matching that pairs a forbidden couple is not a bad matching, it is not a matching at all, and no ranking can express that difference.
- **B** ✓ A stable matching no longer has to be perfect, and once it need not be, the definition has to say what an unmatched person can complain about. The process itself changes by one word: keep asking while there is a free asker who has not asked everybody they are allowed to.
- **C** × One always exists, by the same process on the shortened lists. What changes is the definition of stable, not whether anything satisfies it.
- **D** × Running out of list is exactly what now happens, and it is fine: that person is unmatched and asks no more. The counting argument is untouched, since the lists got shorter rather than longer.

Takeaway. The algorithm survives most variations with one change to the loop. Where the work moves is the definition of what it means to hold, and a definition with four cases instead of one is a definition somebody had to get right.

4.5

Answer: **C**

- **A** × It is the maximum matching, and Priya and MIT break it: Priya would rather have MIT than NYU, and MIT would rather have Priya than Sam. Stability is a test of what people want, and the number of pairs is not part of it. Whether a matching with nobody left out exists at all depends only on which pairs are allowed, which is what CLRS Exercise 26.3-4 (Hall's theorem) is about, and it says nothing about anybody's list.
- **B** × One exists, the same way it always does: run the process on the shortened lists. Two people left with nobody only break a matching if they are allowed each other, and Sam and NYU are not.
- **C** ✓ Priya and MIT are first on each other's lists, so any matching that keeps them apart is broken by them, the maximum one included. Once they are together nobody can complain: Sam is allowed only MIT, which would rather keep Priya, and NYU is allowed only Priya, who would rather keep MIT. Sam and NYU both have nobody, but they are not allowed each other, so that is not a complaint either.
- **D** × Only one holds, so both runs end at it. With the students asking, Priya is held by MIT, her first choice, and Sam's one ask is MIT, which would rather have Priya. With the schools asking, MIT is held by Priya, and NYU's one ask is Priya, who would rather have MIT. Two asks each way, in any order, and Priya with MIT both times.

Takeaway. Largest and stable are different questions about the same lists. The matchings in CLRS section 26.3 come with no preferences at all, so the best one is simply the biggest. Here every pair is two people with opinions, and the stable matching can be smaller than a maximum one. It is never smaller than half, though: a stable matching never leaves two people with nobody when they are allowed each other, so no pair can be added to it, and CLRS Problem 35-4 asks you to prove that a matching nothing can be added to has at least half as many pairs as a maximum one.

4.6

Answer: **C**

- **A** × One always exists. Some students end with nothing, by arithmetic, and the definition of stable is extended to cover them: an unmatched student and a hospital that would rather have them than somebody it has is an instability.
- **B** × The students set aside are exactly where the instability would live. One of them may be preferred by a hospital to somebody it took, and the definition has a case for precisely that.
- **C** ✓ A hospital with q posts is q hospitals with one post and the same list, so splitting it turns the variant into the ordinary problem and the ordinary proofs come along unchanged. Holding q at a time is how that is implemented rather than a different algorithm.
- **D** × Fixing the over-subscribed hospitals is the entire problem rather than a tidying step, and nothing about this start makes the fixing easier.

Takeaway. This variant came first. The National Resident Matching Program had been running a version of this algorithm for ten years when Gale and Shapley published in 1962, and the generalisation costs one line, because a hospital with q posts is q hospitals with one.

4.7

True: **B** · False: A, C, D, E, F

- **A** × Priya would rather have MIT than NYU, but MIT ranks Priya below Ravi, who it has. MIT would rather stay, so this pair is neither kind.
- **B** ✓ Priya would rather have UMass Amherst than NYU, and UMass Amherst ranks Priya above Sam, who it has. Both strictly.
- **C** × Sam would rather have MIT than UMass Amherst, but MIT ranks Sam level with Ravi. Not minding is not the same as wanting, so this one is weak and not strong.
- **D** × Sam would rather have NYU than UMass Amherst, and NYU ranks all three students level. Weak, not strong.
- **E** × Ravi ranks MIT and UMass Amherst level, and UMass Amherst would rather keep Sam. Neither kind.
- **F** × Ravi would rather have NYU than MIT, and NYU ranks everybody level. Weak, not strong.

Takeaway. A strong instability needs both people to strictly want the switch. A tie on either side is enough to stop a pair being strong, and it is not enough to stop it being weak, which is what the next question asks about.

4.8

True: **B, C, D, F** · False: A, E

- **A** × Priya strictly wants MIT, but MIT strictly prefers Ravi, who it has. For weak, the other side has to want the switch or not mind it, and MIT minds.
- **B** ✓ The strong one from the last question. Both strictly want the switch, which more than meets the weak definition.
- **C** ✓ Sam strictly prefers MIT to UMass Amherst, and MIT has Sam level with Ravi. One strict, one indifferent.
- **D** ✓ Sam strictly prefers NYU to UMass Amherst, and NYU has everybody level. One strict, one indifferent.
- **E** × Ravi does not mind, but nobody here strictly wants anything: UMass Amherst would rather keep Sam. Weak needs one side to strictly want it.
- **F** ✓ Ravi strictly prefers NYU to MIT, and NYU has everybody level. One strict, one indifferent.

Takeaway. Every strong instability is also a weak one, so the weak list starts from the strong one and adds each pair where one side strictly wants the switch and the other does not mind. Four here, against one strong.

4.9

Answer: **B**

- **A** × Sam and MIT break it weakly: Sam would rather have MIT than NYU, and MIT has Sam level with Ravi, who it has. So do Ravi and NYU.
- **B** ✓ No pair has both people strictly wanting the switch. Sam and MIT is one of two weak ones; Ravi and NYU is the other.
- **C** × Sam strictly wants MIT, but MIT ranks Sam level with Ravi. Level is not better, so the pair is weak and not strong.
- **D** × No matching can be that: a strong instability meets the weak definition too, so a matching with a strong one always has a weak one.

Takeaway. Free of strong instabilities and free of weak ones are two different checks, and this matching passes one and fails the other. Read which one a question asks for before answering it.

4.10

Answer: **C**

- **A** × Priya does strictly prefer MIT to UMass Amherst, but MIT ranks Priya level with Ravi, who it has. Strong needs MIT to strictly prefer her.
- **B** × Gaining is not the test for weak. MIT does not mind and Priya strictly wants the switch, which is exactly what weak asks for.
- **C** ✓ Priya strictly prefers MIT to UMass Amherst; MIT ranks Priya level with Ravi. One strict, one indifferent.
- **D** × Her tie is between UMass Amherst and NYU, and MIT is above both. A tie only matters when it is between the two people being compared.

Takeaway. Indifference counts toward weak and never toward strong. The strong one in this matching is somewhere else: Sam and MIT, where each strictly prefers the other.

4.11

True: **A, C, F** · False: B, D, E

- **A** ✓ No pair both strictly want the switch. It is not free of weak ones: Sam and MIT, and Ravi and MIT, are both weak.
- **B** × Sam and MIT: Sam would rather have MIT than NYU, and MIT would rather have Sam than Priya. Strong.
- **C** ✓ Nothing breaks it, of either kind. Sam has a school from his top group, and Priya and Ravi would each rather have MIT, which would rather keep Sam.
- **D** × The matching from the last question. Sam and MIT each strictly prefer the other: Sam to NYU, MIT to Ravi.
- **E** × Ravi and NYU: Ravi would rather have NYU than UMass Amherst, and NYU would rather have Ravi than Priya. Strong.
- **F** ✓ No pair both strictly want the switch, though three are weak: Priya with MIT, Priya with UMass Amherst, and Sam with MIT.

Takeaway. A question about every matching of an instance is answered by checking every matching, and with three a side there are only six. The workbench on this page lists them all, with both kinds marked, for any lists you write.

4.12

Answer: **C**

- **A** × Tie-breaking buys the strong notion and not the weak one. Breaking a tie in favour of one person stops the other from blocking in the strict instance, and they are still blocking weakly in the original.
- **B** × The process cannot loop: break the ties any way at all and it is the ordinary process on strict lists, which stops in at most n^2 asks.
- **C** ✓ For the first, break the ties arbitrarily and run the process: a pair who both strictly prefer each other in the original also do so after any tie-break, so no strongly blocking pair survives. For the second, take two students who both rank MIT above NYU, with both schools indifferent between them. Whichever way they are paired, the student at NYU strictly prefers MIT and MIT is indifferent, so every matching has a weakly blocking pair.
- **D** × The strong one always exists, whichever way the ties are broken. And the counterexample for the weak one defeats every matching of that instance, so no tie-break can rescue it either.

Takeaway. Ties are the expensive relaxation, and the cost lands on the notion that survives. Matchings with no strongly blocking pair always exist, and once lists may also be incomplete they can differ in size, at which point finding a largest one is NP-hard, meaning it is one of the problems nobody knows how to solve quickly. The other notion is the opposite: a matching with no weakly blocking pair need not exist at all, and when one does exist it can be found quickly, with the work growing like a power of n rather than doubling with each extra person. That is why every clean statement on this page begins by assuming strict and complete lists.